

Prediksi Kondisi Kesehatan Baterai Lithium-Ion Menggunakan Decision Tree pada NASA Battery Dataset



Dosen :

Kusnawi, S.Kom., M.Eng.

disusun oleh :

Muhamad Fikih Rizaldi

23.11.5724

**FAKULTAS ILMU KOMPUTER
UNIVERSITAS AMIKOM YOGYAKARTA
YOGYAKARTA**

2026

1. LATAR BELAKANG PENELITIAN

Baterai lithium-ion merupakan salah satu komponen energi yang banyak digunakan pada perangkat elektronik, kendaraan listrik, sistem industri, dan penyimpanan energi. Keunggulan baterai ini terletak pada densitas energi yang tinggi dan umur pakai yang relatif panjang. Meskipun demikian, penggunaan berulang akan menyebabkan kapasitas baterai menurun secara bertahap. Penurunan kapasitas tersebut dapat memengaruhi performa perangkat dan meningkatkan risiko gangguan operasional.

Kondisi kesehatan baterai umumnya dinilai melalui State of Health (SOH), yaitu perbandingan antara kapasitas aktual dengan kapasitas awal atau nominal. Apabila SOH terus menurun, baterai dapat masuk ke kondisi Warning atau End of Life. Oleh karena itu, diperlukan metode yang mampu membantu mengklasifikasikan kondisi baterai berdasarkan data pengukuran sensor.

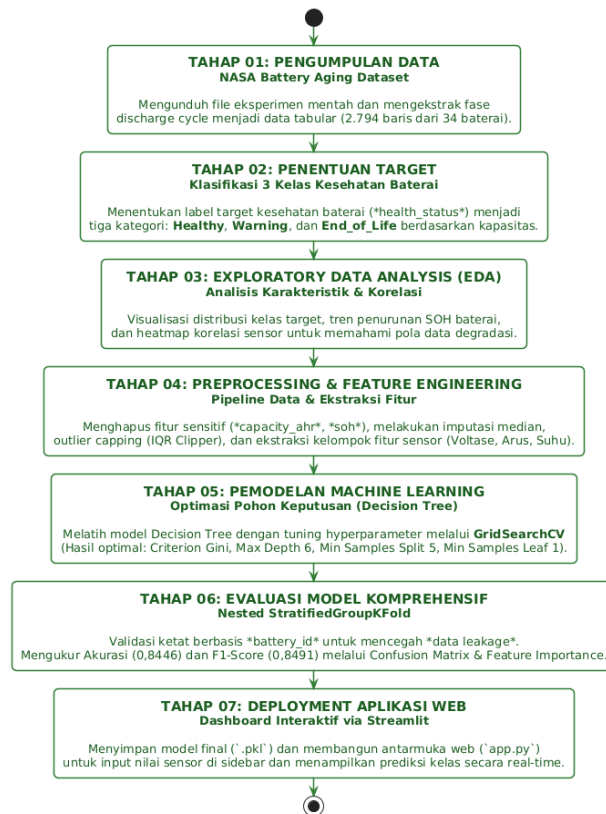
Dataset yang digunakan dalam penelitian ini adalah NASA Battery Dataset. Dataset tersebut berisi data eksperimen baterai lithium-ion dalam proses charge, discharge, dan impedance. Pada penelitian ini, fokus diarahkan pada data discharge karena data tersebut memuat informasi kapasitas dan pola sensor yang relevan untuk melihat degradasi baterai.

Model yang digunakan adalah Decision Tree. Model ini dipilih karena hasilnya lebih mudah dijelaskan dibandingkan beberapa model machine learning lain. Struktur pohon keputusan dapat dianalisis melalui aturan keputusan sehingga sesuai untuk laporan akademik. Namun, Decision Tree memiliki risiko overfitting apabila kedalaman pohon tidak dikontrol. Oleh karena itu, penelitian ini menggunakan pembatasan hyperparameter, GridSearchCV, dan validasi berbasis battery_id.

Klaim hasil penelitian ini dibatasi pada data hasil ekstraksi NASA Battery Dataset yang digunakan dalam proyek. Model tidak dimaksudkan untuk langsung digeneralisasi ke semua jenis baterai lithium-ion di dunia nyata tanpa pengujian tambahan, karena karakteristik degradasi baterai sangat dipengaruhi oleh kondisi operasional, desain sel, suhu, dan pola penggunaan.

2. DIAGRAM ALUR PENELITIAN

Diagram alur penelitian ditampilkan pada Gambar 1 berikut ini. Diagram tersebut menunjukkan tahapan penelitian mulai dari pengumpulan dataset hingga deployment aplikasi web.



Gambar 1. Diagram Alur Penelitian

Alur penelitian dimulai dari pengumpulan NASA Battery Dataset dalam format ZIP dan MAT. Data kemudian diekstraksi untuk mengambil siklus discharge dan membentuk dataset fitur `battery_features.csv`. Setelah dataset terbentuk, dilakukan EDA untuk memahami distribusi kelas, pola SOH, dan hubungan antar fitur.

Tahap berikutnya adalah preprocessing dan feature engineering. Pada tahap ini, fitur yang terlalu dekat dengan target seperti `capacity_ahr` dan `soh` tidak digunakan sebagai input model. Model Decision Tree kemudian dilatih dan dievaluasi menggunakan nested StratifiedGroupKFold berdasarkan `battery_id`. Setelah model akhir diperoleh, model disimpan dalam format `pkl` dan digunakan pada dashboard web berbasis Streamlit.

3. LAMPIRAN DAN ANALISIS KODE PROGRAM

3.1 Pengambilan Data

Dataset yang digunakan dalam eksperimen ini sepenuhnya bersumber dari NASA Randomized Battery Dataset yang disediakan oleh Prognostics Center of Excellence (PCoE). Secara spesifik, penelitian ini mengekstraksi data operasional dari empat kelompok sel baterai Lithium-Ion tipe 18650, yaitu baterai dengan identitas operasional B0005, B0006, B0007, dan B0018.

Fokus utama pemrosesan diarahkan pada lembar kerja pengosongan (discharge) karena pada fase inilah indikator nyata penuaan baterai dapat diamati melalui penurunan kapasitas nominal secara konvensional. Setelah melalui proses agregasi berbasis siklus

(cycle-based aggregation) dari file mentah, total rekaman data bersih yang berhasil dihimpun dan siap diumpankan ke dalam algoritma berjumlah 2.794 baris observasi yang mencerminkan fase penurunan performa baterai dari kondisi awal hingga mencapai titik kritis penuaan (End of Life).

Inventarisasi Dataset dan Pembacaan Data

```
df = pd.read_csv(DATA_PATH)
df = df.drop_duplicates()
df = df.dropna(subset=[TARGET, GROUP_COLUMN])

drop_columns = [
    GROUP_COLUMN,
    TARGET,
    "capacity_ahr",
    "soh",
    "elapsed_time_sec",
    "cycle_index",
    "discharge_index",
]
feature_columns = [col for col in df.columns if col not in drop_columns]
X = df[feature_columns]
y = df[TARGET]
groups = df[GROUP_COLUMN]
```

Kode tersebut membaca dataset hasil ekstraksi, menghapus data duplikat, memastikan target dan group tersedia, serta memisahkan kolom fitur dan target. Kolom capacity_ahr dan soh tidak digunakan sebagai fitur karena keduanya berkaitan langsung dengan pembentukan label health_status.

3.2 Exploratory Data Analysis (EDA)

EDA dilakukan untuk memahami karakteristik dataset sebelum proses pemodelan. Analisis ini mencakup distribusi kelas health_status, tren SOH, dan korelasi antar fitur numerik. Hasil EDA disimpan pada folder outputs/01_eda.

Visualisasi EDA

```
# Tahap 2: eksplorasi data
def make_eda_plots(df):
    EDA_DIR.mkdir(parents=True, exist_ok=True)

    fig, ax = plt.subplots(figsize=(7, 4))
    sns.countplot(data=df, x=TARGET, order=df[TARGET].value_counts().index, ax=ax)
    ax.set_title("Distribusi Kelas Kondisi Baterai")
    ax.set_xlabel("Status Kesehatan")
    ax.set_ylabel("Jumlah Data")
    fig.tight_layout()
    fig.savefig(EDA_CLASS_PATH, dpi=180)
    plt.close(fig)

    fig, ax = plt.subplots(figsize=(9, 5))
    sample_batteries = sorted(df[GROUP_COLUMN].unique())[:8]
```

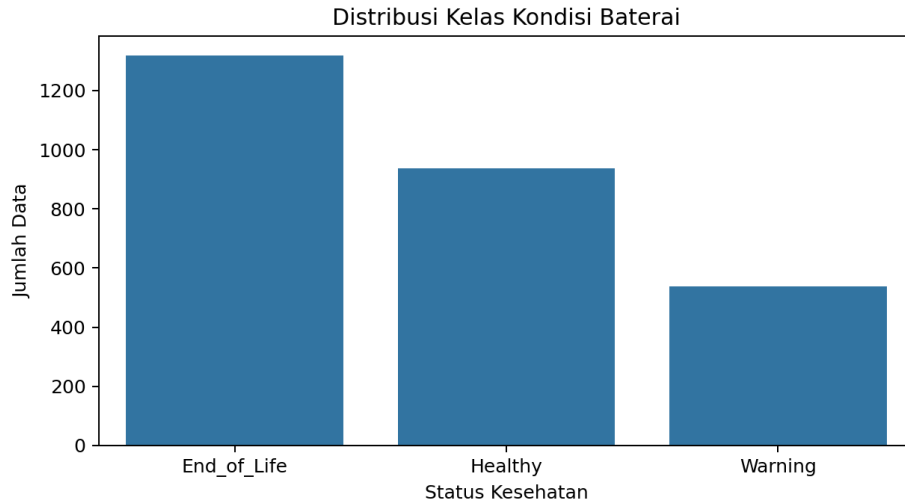
```

sns.lineplot(
    data=df[df[GROUP_COLUMN].isin(sample_batteries)],
    x="discharge_index",
    y="soh",
    hue=GROUP_COLUMN,
    ax=ax,
)
ax.axhline(0.8, color="orange", linestyle="--", label="Batas Warning")
ax.axhline(0.7, color="red", linestyle="--", label="Batas End of Life")
ax.set_title("Tren Penurunan State of Health")
ax.set_xlabel("Siklus Discharge")
ax.set_ylabel("State of Health")
fig.tight_layout()
fig.savefig(EDA_SOH_PATH, dpi=180)
plt.close(fig)

corr_columns = [
    "cycle_index",
    "discharge_index",
    "elapsed_time_sec",
    "voltage_mean",
    "voltage_min",
    "current_mean",
    "temperature_mean",
    "temperature_max",
    "capacity_ahr",
    "soh",
]
fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(df[corr_columns].corr(numeric_only=True), annot=True, fmt=".2f",
cmap="vlag", ax=ax)
ax.set_title("Korelasi Fitur Numerik")
fig.tight_layout()
fig.savefig(EDA_CORR_PATH, dpi=180)
plt.close(fig)

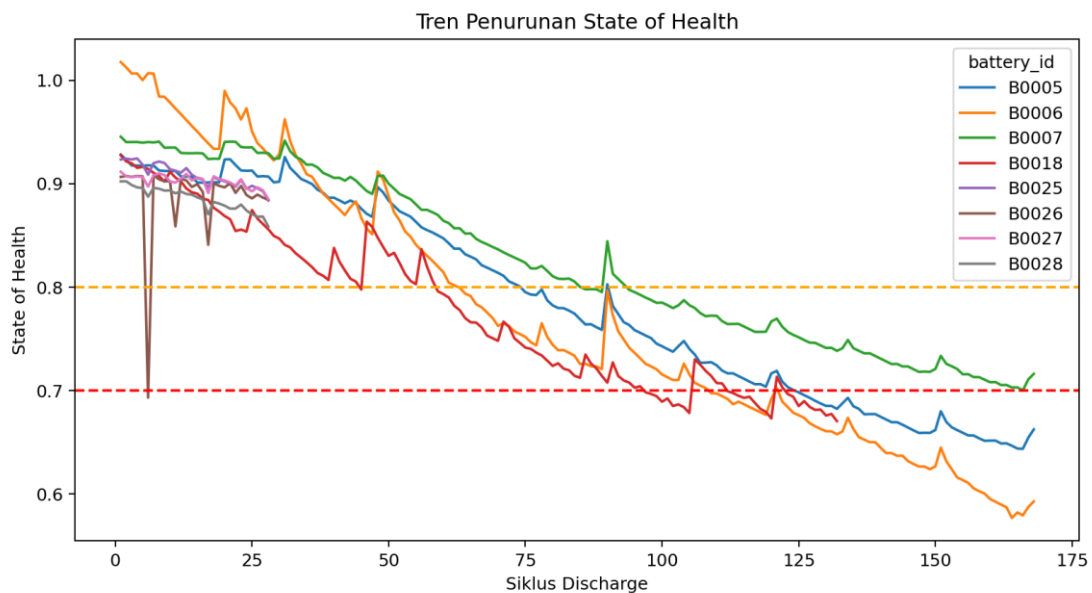
with EDA_NOTE_PATH.open("w", encoding="utf-8") as file:
    file.write("Catatan EDA\n\n")
    file.write("Grafik SOH dan korelasi kapasitas dipakai hanya untuk memahami\n")
    file.write("pola data.\n")
    file.write("Kolom capacity_ahr dan soh tidak digunakan sebagai fitur saat\n")
    file.write("melatih model.\n")

```



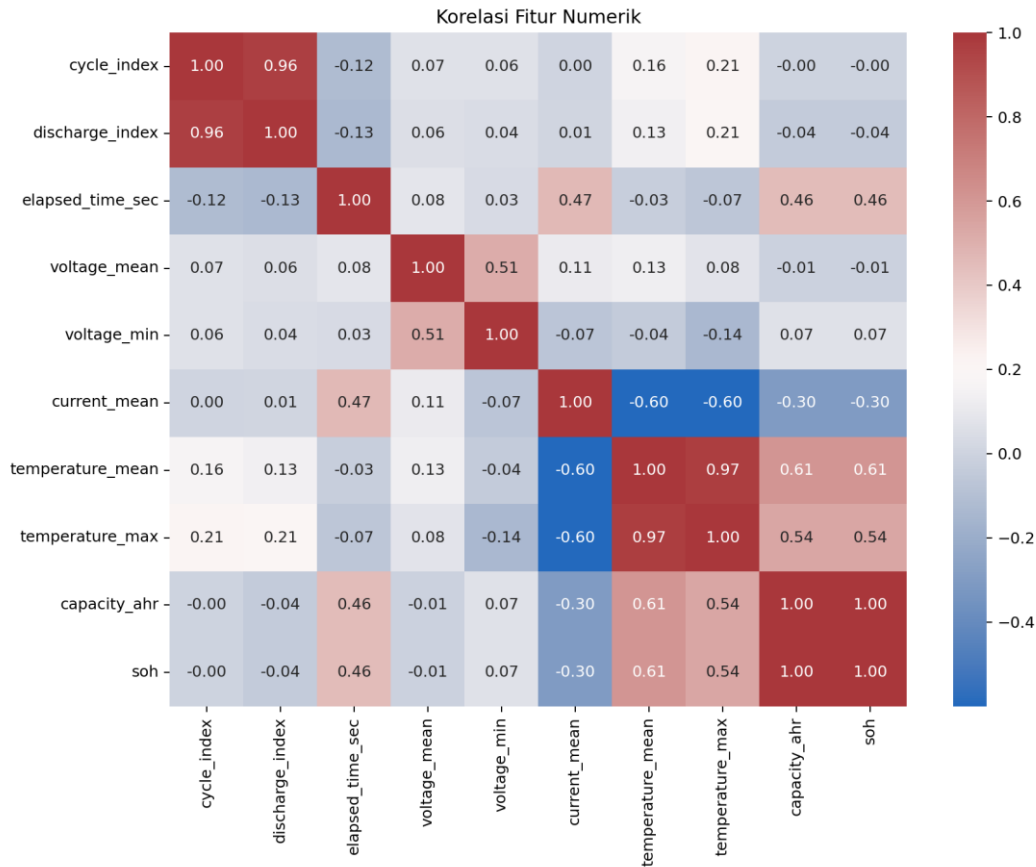
Gambar 2. Distribusi Kelas Health Status

Visualisasi pada Gambar 2 menampilkan proporsi jumlah sampel data untuk setiap kategori target status kesehatan baterai, yaitu Healthy, Warning, dan End of Life melalui grafik batang hitung (countplot). Grafik ini mengindikasikan adanya fenomena ketidakseimbangan kelas (class imbalance), di mana jumlah rekaman data untuk setiap fase degradasi memiliki frekuensi kemunculan yang berbeda secara signifikan. Fase Warning sebagai fase transisi penuaan sel elektrokimia umumnya memiliki rentang waktu siklus yang lebih pendek dalam eksperimen fisik, sehingga menghasilkan populasi data yang lebih sedikit dibandingkan kelas dominan lainnya. Temuan dari visualisasi distribusi ini mendasari keputusan teknis pada konfigurasi algoritma Decision Tree di tahap berikutnya, di mana parameter penyeimbang bobot kelas otomatis (`class_weight="balanced"`) wajib diaktifkan agar model tidak bias pada kelas mayoritas dan tetap sensitif dalam mengklasifikasikan kondisi transisi kritis baterai.



Gambar 3. Tren SOH Baterai

Gambar 3 memvisualisasikan dinamika penurunan nilai State of Health (SOH) secara progresif dari sampel delapan kelompok baterai utama seiring dengan bertambahnya indeks urutan pengosongan (discharge_index) melalui visualisasi diagram garis (lineplot). Di dalam grafik runtun waktu penuaan ini, diintegrasikan dua garis ambang batas horizontal kritis sebagai acuan dasar pembentukan label kondisi, yaitu garis pembatas putus-putus warna oranye (kondisi Warning pada batas nilai SOH 0.8) dan garis pembatas putus-putus warna merah (kondisi End of Life pada batas nilai SOH 0.7). Tren penurunan yang konsisten dan searah pada grafik ini membuktikan secara empiris terjadinya efek penuaan akumulatif (aging effect) pada sel baterai yang tidak dapat dihindari, sekaligus memberikan validasi visual bahwa data sensor operasional berkorelasi erat terhadap deret waktu penurunan kapasitas energi baterai.



Gambar 4. Heatmap Korelasi Fitur

Matriks korelasi pada Gambar 4 memetakan kekuatan hubungan linear antar variabel sensor operasional utama yang diekstraksi dari proses discharge (seperti tegangan, arus, indeks siklus, durasi waktu, dan suhu) bersama dengan parameter kapasitas (capacity_ahr) dan SOH (soh). Berdasarkan visualisasi heatmap dengan skema warna kontras ini, ditemukan informasi krusial di mana variabel kapasitas dan SOH menunjukkan nilai koefisien korelasi yang sangat kuat dan bernilai mutlak mendekati angka 1.00 atau -1.00 terhadap penentuan kelas target kesehatan baterai.

Berdasarkan temuan visualisasi matriks korelasi dan luaran dokumen teks pembatasan data (catatan_eda.txt), diambil keputusan arsitektur data yang mutlak untuk proses pemodelan. Kolom variabel capacity_ahr dan soh bersama dengan beberapa fitur penanda indeks (elapsed_time_sec, cycle_index, discharge_index) wajib dikeluarkan dari daftar fitur masukan (EXCLUDED_COLUMNS) dan dilarang keras digunakan untuk melatih model Decision Tree. Jika fitur-fitur yang menjadi basis perumusan label tersebut tetap diumpungkan ke dalam algoritma saat pelatihan (training), model akan mengalami kebocoran data (data leakage). Akibatnya, pohon keputusan hanya akan menghafal jalan pintas nilai kapasitas tanpa pernah mempelajari karakteristik pola sensor fisik yang sebenarnya terjadi (seperti fluktuasi tegangan, nilai arus minimum, maupun kenaikan suhu maksimum) saat baterai melepaskan energinya. Pembatasan ketat ini menjamin model yang dihasilkan bersifat objektif, kokoh (robust), dan dapat diandalkan saat diimplementasikan pada skenario pengujian dunia nyata.

3.3 Preprocessing

Tahap preprocessing dilakukan untuk memastikan data bersih dari outlier dan nilai yang kosong. Pembersihan data dari outlier ekstrem menggunakan metode Interquartile Range (IQR) dengan batas atas dan bawah $1.5 \times \text{IQR}$ melalui fungsi IQRClipper(). Langkah ini penting agar nilai sensor yang tidak stabil tidak merusak model. Selain itu, untuk menangani nilai kosong (*missing values*), digunakan SimpleImputer dengan strategi median agar distribusi asli data tidak bergeser.

Pipeline Preprocessing dan Model

```
def build_pipeline():
    return Pipeline(
        [
            ("feature_engineering", BatteryFeatureEngineer()),
            ("outlier_capping", IQRClipper(factor=1.5)),
            ("imputer", SimpleImputer(strategy="median")),
            (
                "model",
                DecisionTreeClassifier(random_state=42, class_weight="balanced"),
            ),
        ]
    )
```

Pipeline tersebut memastikan seluruh proses preprocessing dilakukan secara konsisten di setiap fold validasi. Dengan cara ini, proses imputasi dan outlier capping hanya dipelajari dari data latih pada masing-masing fold, sehingga risiko kebocoran data dapat dikurangi.

3.4 Feature Engineering

Feature engineering dilakukan secara konservatif. Fitur turunan yang digunakan berasal dari pola tegangan, arus, dan suhu, seperti voltage_range, current_range, temperature_range, voltage_stability_ratio, dan temperature_stability_ratio. Fitur yang terlalu dekat dengan target tidak digunakan agar model tidak terlihat bagus hanya karena shortcut prediksi.

Kelompok Fitur	Contoh Fitur	Fungsi
Tegangan	voltage_mean, voltage_min, voltage_std	Menggambarkan karakteristik tegangan saat discharge
Arus	current_mean, current_min, current_std	Menggambarkan pola beban dan kestabilan arus
Suhu	temperature_mean, temperature_min, temperature_std	Menggambarkan respons termal baterai
Lingkungan	ambient_temperature	Menunjukkan kondisi suhu eksperimen

3.5 Pemodelan Decision Tree

Model yang digunakan adalah Decision Tree. Untuk proses evaluasi, pengujian tidak menggunakan K-Fold biasa, melainkan menggunakan Nested StratifiedGroupKFold berdasarkan variabel `battery_id`. Hal ini sangat penting untuk mencegah kebocoran data (*data leakage*), sehingga data dari baterai yang sama tidak akan pernah bercampur ke dalam data latih dan data uji secara bersamaan. Pencarian hyperparameter terbaik untuk menentukan kedalaman cabang pohon (*max_depth*) dilakukan secara otomatis menggunakan GridSearchCV.

Hyperparameter Tuning Decision Tree

```
# Konfigurasi Hyperparameter
param_grid = {
    "model__criterion": ["gini", "entropy"],
    "model__max_depth": [3, 4, 5, 6, 7, 8],
    "model__min_samples_split": [2, 5, 10],
    "model__min_samples_leaf": [1, 3, 5, 10],
}

# Implementasi Validasi Nested StratifiedGroupKFold
def run_nested_group_cv(X, y, groups, labels, param_grid):
    outer_cv = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=42)
    inner_cv = StratifiedGroupKFold(n_splits=4, shuffle=True, random_state=42)
    oof_pred = pd.Series(index=X.index, dtype=object)

    for fold, (train_idx, test_idx) in enumerate(outer_cv.split(X, y, groups),
start=1):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train = y.iloc[train_idx]
        groups_train = groups.iloc[train_idx]

        pipeline = build_pipeline()

        # Inner loop: Mencari parameter terbaik menggunakan GridSearchCV
        grid = GridSearchCV(
            estimator=pipeline, param_grid=param_grid,
```

```

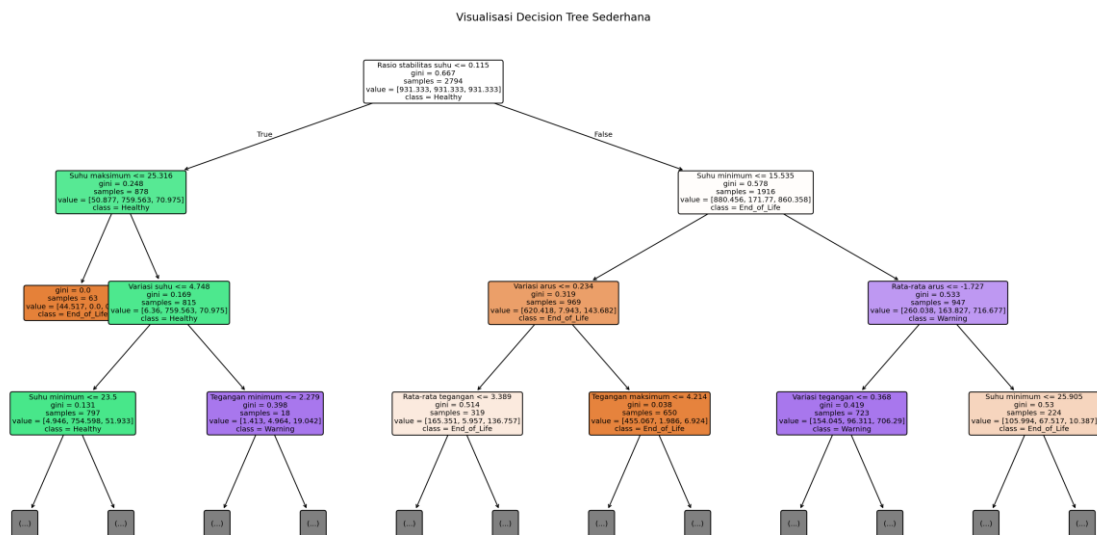
        scoring="f1_weighted", cv=inner_cv, n_jobs=-1,
    )
    grid.fit(X_train, y_train, groups=groups_train)

    # Outer loop: Prediksi pada baterai yang belum pernah dilihat model
    oof_pred.iloc[test_idx] = grid.predict(X_test)

return oof_pred

```

Parameter terbaik model akhir adalah criterion gini, max_depth 6, min_samples_leaf 1, dan min_samples_split 5. Pembatasan max_depth digunakan untuk membantu mengurangi overfitting pada model Decision Tree.

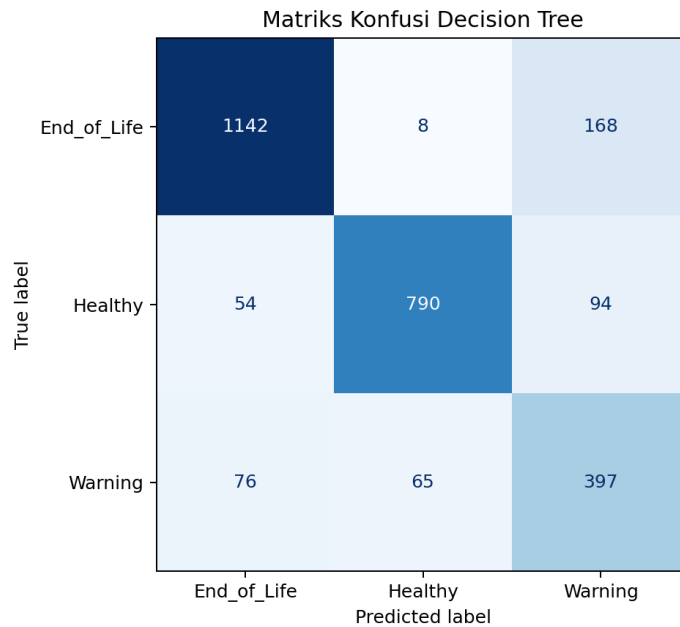


Gambar 5. Struktur Decision Tree Model Akhir

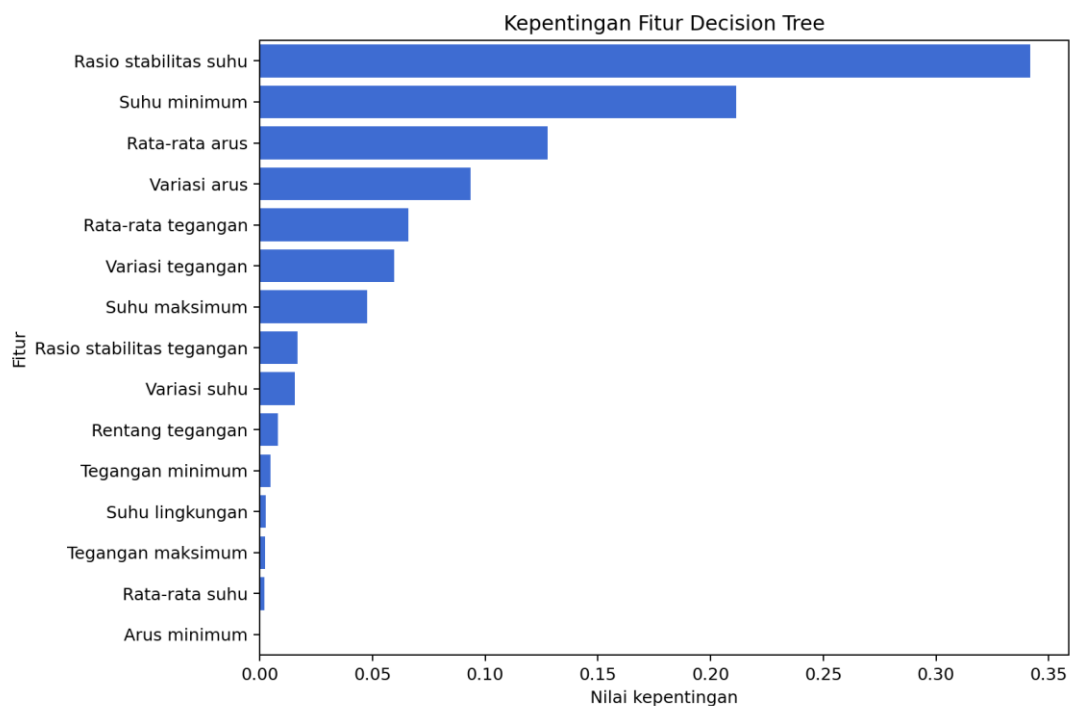
3.6 Evaluasi Model

Evaluasi model dilakukan menggunakan nested StratifiedGroupKFold berdasarkan battery_id. Strategi ini lebih aman dibanding split acak biasa karena data dari baterai yang sama tidak dicampur secara tidak tepat antara data latih dan data uji. Dengan demikian, model diuji pada baterai yang berbeda dari data latih.

Metrik	Nilai
Akurasi	0,8446 +/- 0,0408
F1-score berbobot	0,8491 +/- 0,0445
Precision berbobot	0,8712 +/- 0,0558
Recall berbobot	0,8446 +/- 0,0408
Selisih F1 train-test	0,0976 +/- 0,0539



Gambar 6. Confusion Matrix



Gambar 7. Feature Importance

Berdasarkan classification report out-of-fold, kelas Healthy dan End_of_Life memiliki F1-score yang lebih tinggi dibanding kelas Warning. Kelas Warning lebih sulit diprediksi karena berada pada area transisi antara kondisi baterai sehat dan kondisi baterai yang mendekati akhir masa pakai.

3.7 Ringkasan Hasil Evaluasi

Kelas	Precision	Recall	F1-score	Support
End_of_Life	0,90	0,87	0,88	1318
Healthy	0,92	0,84	0,88	938
Warning	0,60	0,74	0,66	538
Weighted Avg	0,85	0,83	0,84	2794

Hasil tersebut menunjukkan bahwa model cukup baik untuk membedakan kondisi baterai yang sudah jelas, tetapi masih memiliki tantangan pada kelas Warning. Hal ini wajar karena kelas Warning merupakan kondisi peralihan yang secara sensor dapat memiliki kemiripan dengan Healthy maupun End_of_Life.

4. LAMPIRAN DAN ANALISIS DEPLOYMENT WEB

4.1 Deskripsi Singkat Deployment

Model Decision Tree yang telah dilatih dan dievaluasi di-deploy ke dalam aplikasi web berbasis Streamlit. Aplikasi ini dibuat pada file `app.py` dan memuat model dari `models/decision_tree_battery.pkl`. Pengguna dapat memasukkan nilai sensor discharge melalui sidebar, kemudian sistem menampilkan hasil prediksi kondisi baterai.

Menjalankan Aplikasi Web

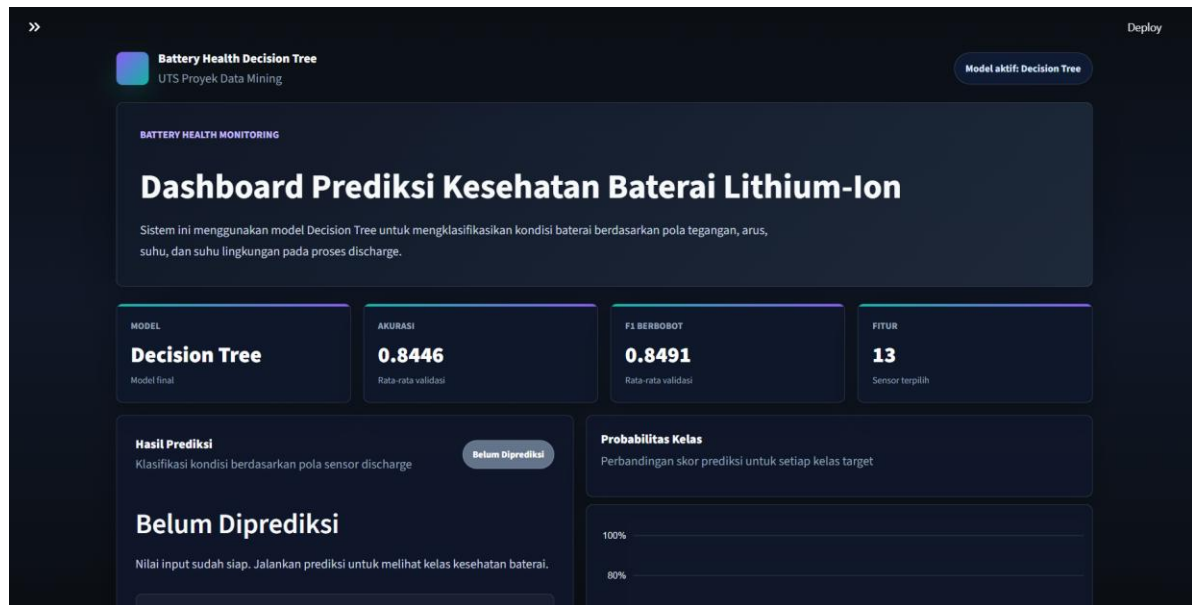
```
streamlit run app.py
```

4.2 Isi Dashboard Web

Isi dashboard web dibagi menjadi beberapa komponen utama. Pertama, dashboard menampilkan ringkasan model, akurasi, F1-score berbobot, dan jumlah fitur yang digunakan. Kedua, terdapat sidebar yang berfungsi sebagai panel input agar pengguna bisa memasukkan nilai sensor baterai secara manual. Ketiga, bagian hasil prediksi akan langsung mengklasifikasikan status baterai ke dalam kelas Healthy, Warning, atau End_of_Life. Terakhir, dashboard juga menyediakan grafik probabilitas kelas untuk melihat tingkat keyakinan model, serta visualisasi feature importance untuk membantu pengguna memahami sensor mana yang paling berpengaruh pada prediksi.

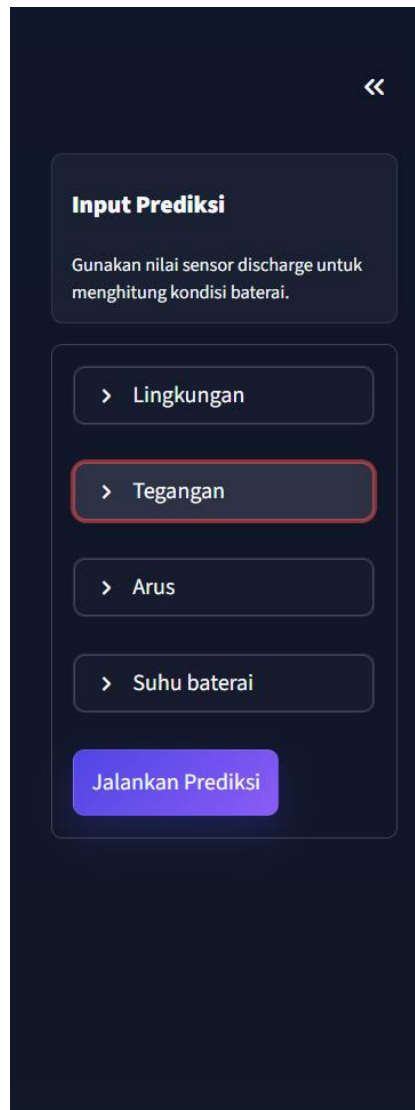
4.3 Screenshot Dashboard

Pada bagian ini, dilampirkan bukti eksekusi dan antarmuka grafis dari aplikasi web *dashboard* pemantauan kesehatan baterai (*Battery Health Monitoring*) yang dibangun menggunakan repositori framework Streamlit. Visualisasi antarmuka ini dibagi menjadi beberapa bagian utama untuk mempermudah interpretasi pengguna.



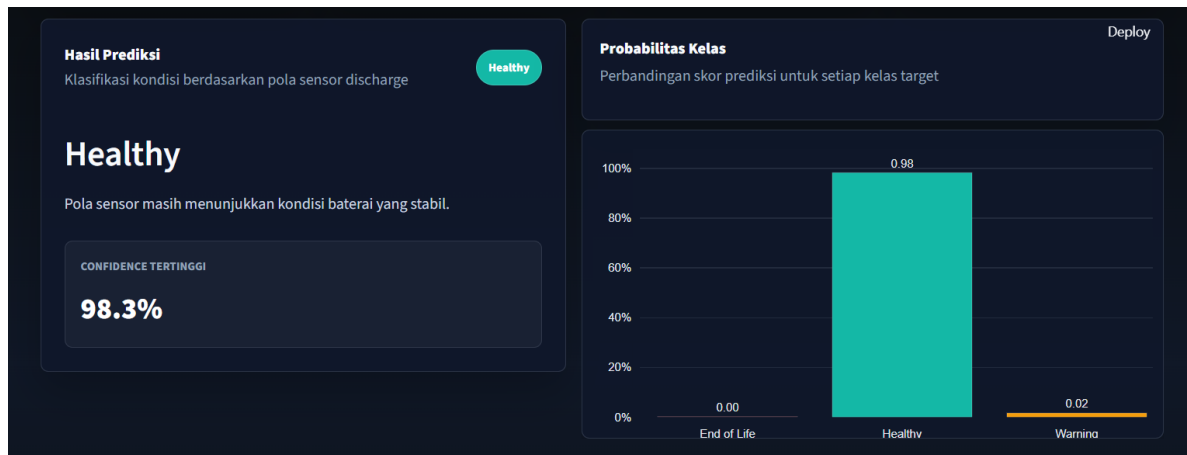
Gambar 8. Tampilan dashboard utama

Gambar 8 menunjukkan halaman utama aplikasi saat pertama kali dieksekusi secara lokal. Pada panel utama, *dashboard* secara transparan menyajikan ringkasan parameter teknis model *Decision Tree* hasil proses *training*, yang meliputi informasi matriks performa global (Nilai Akurasi dan *Weighted F1-score*), jumlah total fitur operasional sensor yang digunakan, serta visualisasi diagram interaktif *Feature Importance* dan performa metrik per *fold* evaluasi.



Gambar 9. Sidebar input prediksi

Gambar 9 menampilkan struktur panel kontrol samping (*sidebar*) yang berfungsi sebagai media interaksi pengguna (*user input interface*). Komponen ini diimplementasikan menggunakan kontrol geser numerik (*numerical slider*) untuk memfasilitasi input nilai aktual dari sensor fisik baterai secara *real-time*, yang mencakup variabel lingkungan (*ambient temperature*), fluktuasi statistik tegangan (*voltage mean*, *voltage min*, *voltage max*, *voltage std*), dinamika arus (*current mean*, *current min*, *current max*, *current std*), dan profil suhu internal sel baterai (*temperature mean*, *temperature min*, *temperature max*, *temperature std*).



Gambar 10. Hasil prediksi dan probabilitas kelas

Gambar 10 menyajikan visualisasi panel luaran (*output panel*) setelah data sensor pada *sidebar* disubmisikan ke dalam mesin inferensi *Decision Tree*. Sistem secara otomatis menampilkan label prediksi status kesehatan baterai (*Healthy*, *Warning*, atau *End of Life*) dengan indikator warna penanda yang tegas. Di bawah label keputusan, diintegrasikan grafik batang horizontal yang merepresentasikan nilai distribusi probabilitas (*class probability distribution*), yang mencerminkan tingkat keyakinan matematis (*confidence level*) algoritma terhadap keputusan klasifikasi yang diambil.

4.4 Analisis Singkat Deployment

Implementasi arsitektur penyebaran model (*model deployment*) berbasis web *dashboard* interaktif menggunakan framework Streamlit ini berhasil menjembatani kesenjangan fungsional antara kompleksitas algoritma *machine learning* di lingkungan *back-end* dengan kebutuhan praktis pengguna di lingkungan *front-end*. Melalui integrasi berkas biner model yang telah diekstraksi (*decision_tree_battery.pkl*), aplikasi mampu melakukan proses inferensi data baru secara langsung (*real-time inference*) tanpa memerlukan ketergantungan terhadap lingkungan komputasi kode pemrograman asal (*training script*).

Sebagai sebuah *prototype* akademik untuk sistem analitik prediktif, platform ini memiliki keunggulan signifikan dalam aspek keterbacaan (*interpretability*). Pengguna non-teknis dapat dengan mudah mengamati dampak perubahan fluktuasi parameter sensor (seperti penurunan tegangan minimum atau lonjakan suhu maksimum) terhadap pergeseran status kesehatan baterai secara visual dan intuitif. Selain itu, penyajian grafik metrik evaluasi per *fold* memberikan transparansi akademis yang membuktikan bahwa model yang ditanamkan telah divalidasi secara ketat menggunakan metode *Nested StratifiedGroupKfold* untuk meminimalkan risiko bias pengujian.

Namun demikian, terdapat beberapa keterbatasan teknis pada fase *deployment* saat ini yang memerlukan pengembangan lebih lanjut. Pertama, operasional aplikasi masih bersifat lokal (*local host execution*) sehingga belum dapat diakses secara luas oleh sistem eksternal sebelum dilakukan migrasi ke layanan *cloud hosting* (seperti Streamlit Community Cloud atau Hugging Face Spaces). Kedua, mekanisme pembaruan model

(*model registry*) belum bersifat dinamis, sehingga apabila terdapat penambahan data eksperimen fisik NASA Battery Dataset yang baru, sistem wajib melalui proses *retraining* dan ekspor ulang berkas biner secara manual untuk memperbarui tingkat akurasi model pada *dashboard*.

5. LAMPIRAN

Github : <https://github.com/FikihRizaldi/Battery-Health-Decision-Tree.git>

Launchinpad : <https://launchinpad.amikom.ac.id/project/battery-health-decision-tree-39d79b0>